Doctoral Dissertation Defense

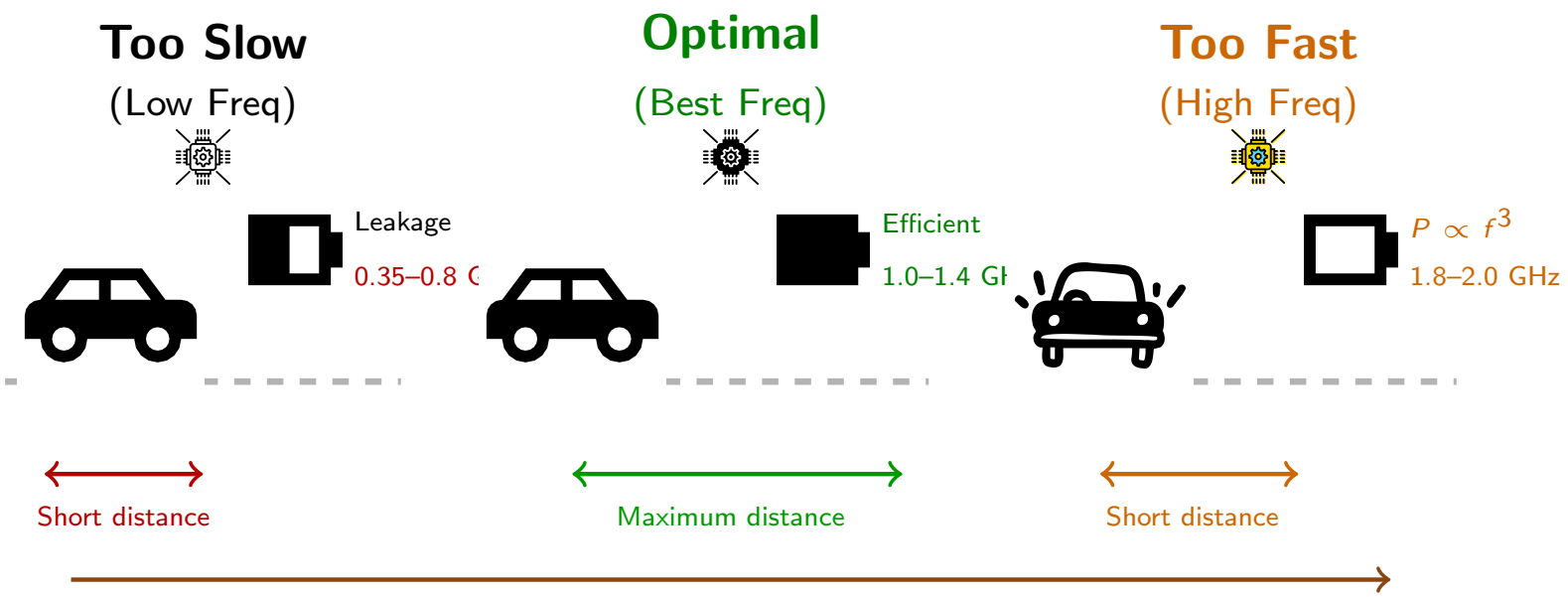
Data-Efficient AI-Guided Energy- and Thermal-Aware Scheduling on Heterogeneous Multicore Systems

Mohammad Pivezhandi
Advisor: Abusayeed Saifullah

Embedded Systems Lab
Department of Computer Science

WAYNE STATE UNIVERSITY

DVFS Intuition: The Car Speed Analogy



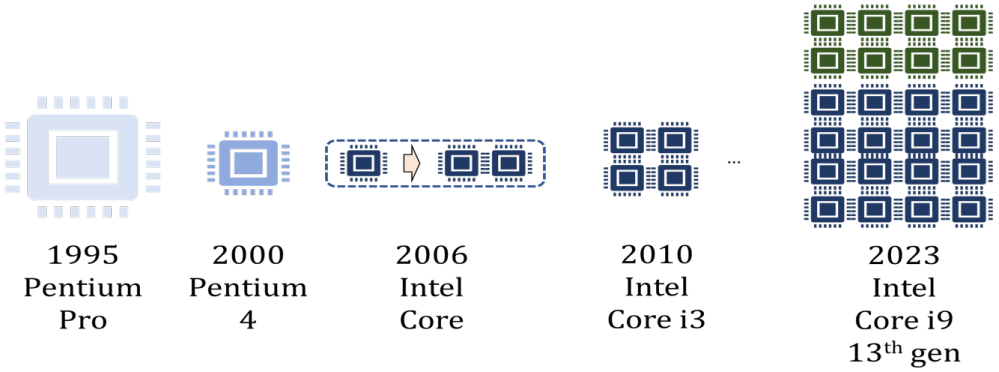
	Gas Car	Tesla Model 3
Optimal Speed	55–65 mph	25–45 mph
Range at Optimal	400+ miles	350+ miles
Range at 80 mph	300 miles	220 miles

Key insight: Like cars, CPUs have an optimal operating point — running too fast wastes energy ($P \propto f^3$)

Frequency ranges shown for NVIDIA Jetson TX2 (345 MHz – 2.035 GHz, 12 levels)

Energy-Aware Scheduling: Motivation

- **Energy Efficiency**
  - Static leakage: 22% → 63% with technology scaling
  - Overheating degrades reliability
  - Cooling costs $3/watt dissipation
- **DVFS and Core Allocation**
  - DVFS reduces power consumption
  - Task-to-core optimizes workload
  - Joint optimization is NP-hard
- **AI-Guided Scheduling**
  - RL adapts to dynamic workloads
  - Challenge: Data efficiency



Static leakage power evolution with technology scaling

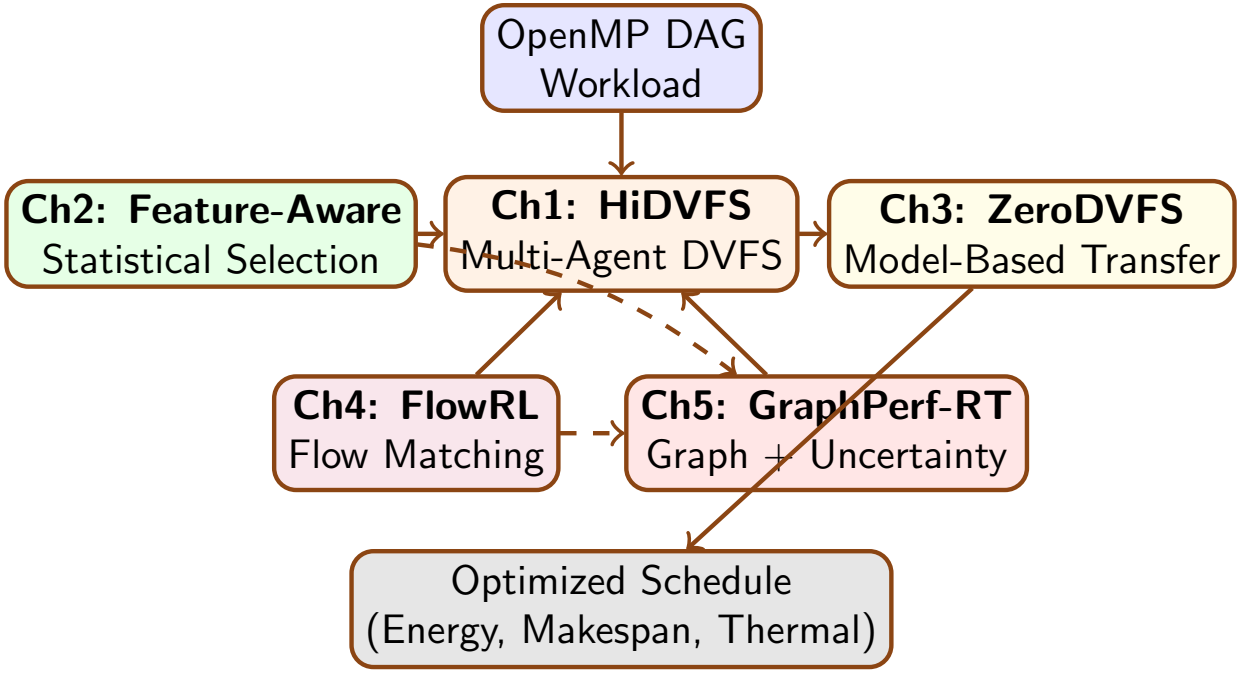
Research Questions (added)

- **RQ1: How to handle large action spaces in multi-core scheduling?**
  - n cores $\times$ m frequencies $\Rightarrow O(m^n)$ combinations
  - *Approach:* Hierarchical multi-agent decomposition (HiDVFS)
- **RQ2: Which features matter most for energy-efficient scheduling?**
  - *Approach:* Statistical feature selection (Feature-Aware)
- **RQ3: How to reduce sample complexity of RL in embedded systems?**
  - *Approach:* Learned environment models (ZeroDVFS)
- **RQ4: How to generate realistic synthetic data for few-shot RL?**
  - *Approach:* Distribution-aware flow matching (FlowRL)
- **RQ5: How to provide calibrated uncertainty for safe scheduling?**
  - *Approach:* Evidential deep learning on graphs (GraphPerf-RT)

Dissertation Contributions

- **HiDVFS** – Hierarchical Multi-Agent DVFS (Dynamic Voltage and Frequency Scaling) Scheduler
  - Model-based reward prediction with per-cluster temperature profiling
- **Feature-Aware Allocation** – Statistical Learning for Task Allocation
  - Statistical feature selection for energy- and thermal-aware task-to-core mapping
- **ZeroDVFS** – Model-Based Few-Shot Learning
  - Learned environment models for zero-shot transfer across platforms
- **FlowRL** – Flow Matching for Synthetic Data
  - Distribution-aware flow matching for data augmentation in few-shot RL (Reinforcement Learning)
- **GraphPerf-RT** – Graph-Based Performance Prediction
  - Heterogeneous graph with evidential uncertainty for calibrated predictions

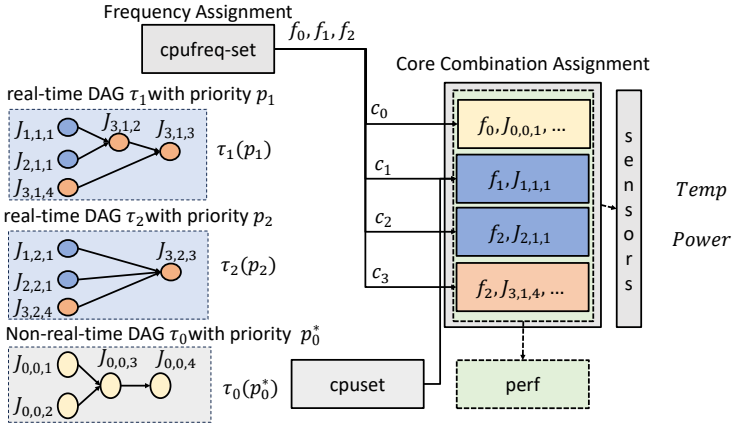
Unified Framework: How Contributions Connect (added)



Feature-Aware → identifies critical features
HiDVFS → multi-agent scheduling
ZeroDVFS → cross-platform transfer
FlowRL → generates synthetic data
GraphPerf-RT → uncertainty-aware prediction

Energy-Aware Scheduling: Example

- **OpenMP (Open Multi-Processing) DAG (Directed Acyclic Graph) Workloads**
  - Tasks with dependencies form DAGs
  - Irregular execution patterns
  - Branch misses, cache behavior vary
- **Scheduling Decisions**
  - Which cores to allocate?
  - What frequency per core?
  - How to prioritize tasks?
- **Optimization Goals**
  - Minimize makespan
  - Minimize energy consumption
  - Maintain thermal constraints



DAG task model with core allocation

Comparison with Prior Work

Work	Adaptive	Thermal	Multi-Core	Data-Eff.	Transfer
Linux Governors	×	×	✓	N/A	×
Table-Based [Pagani et al. TCAD'18]	×	✓	✓	×	×
Single-Agent RL [Yu et al. Electronics'20]	✓	×	×	×	×
GearDVFS [Lin et al. MobiCom'23]	✓	✓	✓	×	×
zTT [Kim et al. MobiSys'21]	✓	✓	✓	×	×
Our Work	✓	✓	✓	✓	✓

- Linux governors: ondemand, schedutil – ignore thermal constraints
- Table-based: 8-12 hours offline profiling, no runtime adaptation
- GearDVFS/zTT: Model-free RL, thousands of episodes to converge

Prior Work: Detailed Comparison

- **Our Work**: Online, energy/thermal-aware, real-time, few-shot RL

Work	Year	Online	Energy	Thermal	Real-time	Multi-obj. RL	Few Shot
[Wang et al. ASP-DAC'17]	2017	×	✓	×	×	×	×
[Pagani et al. TCAD'18]	2018	✓	✓	✓	✓	✓	×
[Dinakarrao et al. JETC'19]	2019	×	✓	✓	×	✓	×
[Yu et al. Electronics'20]	2020	✓	✓	×	×	×	×
[Kim et al. MobiSys'21]	2021	✓	✓	✓	×	✓	×
[Lin et al. MobiCom'23]	2023	✓	✓	✓	×	✓	×
[Zhang et al. TMC'24]	2024	✓	✓	×	×	✓	×
Our Work	–	✓	✓	✓	✓	✓	✓

Challenge 1: Large Action Space

- **Exponential Complexity**
  - n cores, m frequency levels $\Rightarrow O(m^n)$ combinations
  - Jetson TX2: 6 cores, 12 frequencies $= 12^6 \approx 3$ million actions
  - Intel Core i7: 14 cores $\Rightarrow$ even larger space
- **Prior Approaches**
  - Single-agent RL struggles with high-dimensional spaces
  - Heuristics lack adaptability
- **Our Solution: Hierarchical Multi-Agent RL**
  - Decompose into Profiler + Temperature + Priority agents
  - Reduces to $O(m \times n)$ per agent

Challenge 2: Data Efficiency

- **Sample Complexity of RL**
  - Model-free RL requires thousands of episodes
  - Each episode = real hardware execution
  - Impractical for embedded systems
- **Prior Approaches**
  - Offline profiling: 8–12 hours of table generation
  - Cannot adapt at runtime
- **Our Solutions**
  - Model-based RL with environment models (ZeroDVFS)
  - Flow matching for synthetic data (FlowRL)
  - Converge in 3–12 episodes vs. hundreds

Challenge 3: Platform Heterogeneity

- **Diverse Hardware Platforms**
  - ARM (Advanced RISC Machines): Jetson TX2, Orin NX, RUBIK Pi
  - x86 (Intel Core i7, Xeon)
  - Different core counts, frequency ranges, thermal behavior
- **Prior Approaches**
  - Platform-specific policies don't transfer
  - Retrain from scratch for each platform
- **Our Solutions**
  - Transfer learning across platforms (ZeroDVFS)
  - Graph-based representation for generalization (GraphPerf-RT)

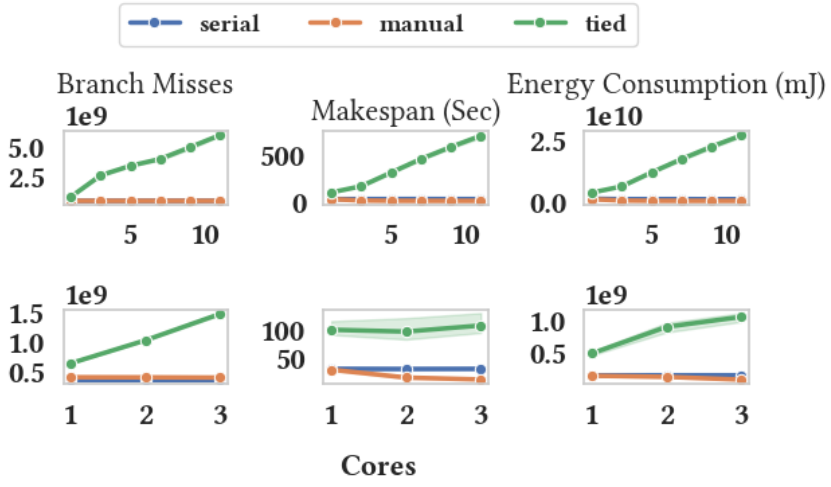
Platform	Cores	Freq. Levels	Architecture
Jetson TX2	6	12	ARM Denver + Cortex
Jetson Orin NX	8	14	ARM Cortex-A78AE
RUBIK Pi	8	8	ARM Cortex-A76/A55
Intel Core i7	14	16	x86 P-cores + E-cores

Challenge 4: Workload Irregularity

- **OpenMP DAG Characteristics**
  - Unpredictable task execution times
  - Data-dependent control flow
  - Cache/branch behavior varies
- **Impact on Scheduling**
  - Static schedules suboptimal
  - Runtime features essential
- **Our Solution**
  - Feature-aware allocation
  - Runtime profiling integration

	Jetson TX2	Jetson Orin NX	RUBIK Pi
Cores	6	8	8
Thermal Zones	8	9	36
FFT† (s)	1.13	0.89	1.45
Strassen (s)	2.41	1.92	3.12

† FFT = Fast Fourier Transform



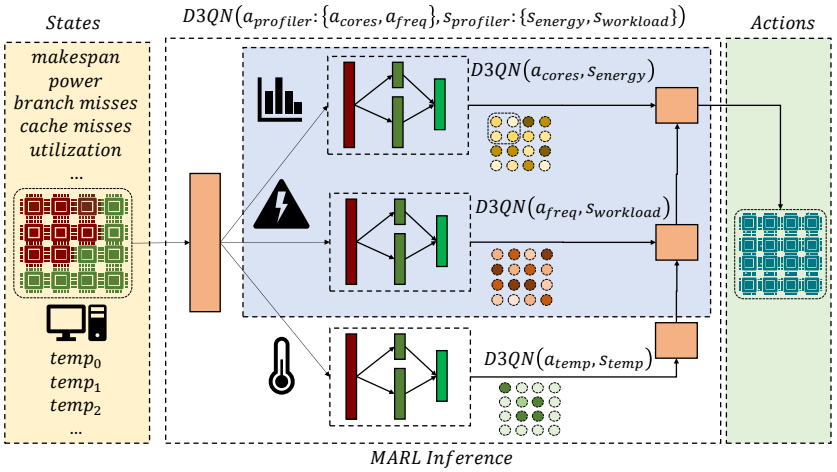
N-Queens on Xeon (12 cores) and Core i7 (4 cores). More cores increase branch misses for tied tasks.

Challenge 5: Uncertainty Quantification

- **Prediction Reliability**
  - Performance models can be wrong
  - Wrong predictions lead to thermal violations
  - Need confidence estimates for safety-critical decisions
- **Prior Approaches**
  - Point predictions without uncertainty
  - Ensemble methods are expensive
- **Our Solution: Evidential Deep Learning**
  - Normal-Inverse-Gamma priors for calibrated uncertainty
  - Single forward pass uncertainty estimation
  - Filter low-confidence predictions (GraphPerf-RT)

Why Reinforcement Learning?

- **Adaptive Decision Making**
  - Learn optimal policies from experience
  - No need for explicit system models
- **Multi-Objective Optimization**
  - Balance energy, makespan, temperature
  - Reward shaping for complex objectives
- **Runtime Adaptation**
  - Responds to changing workloads
  - Handles thermal dynamics



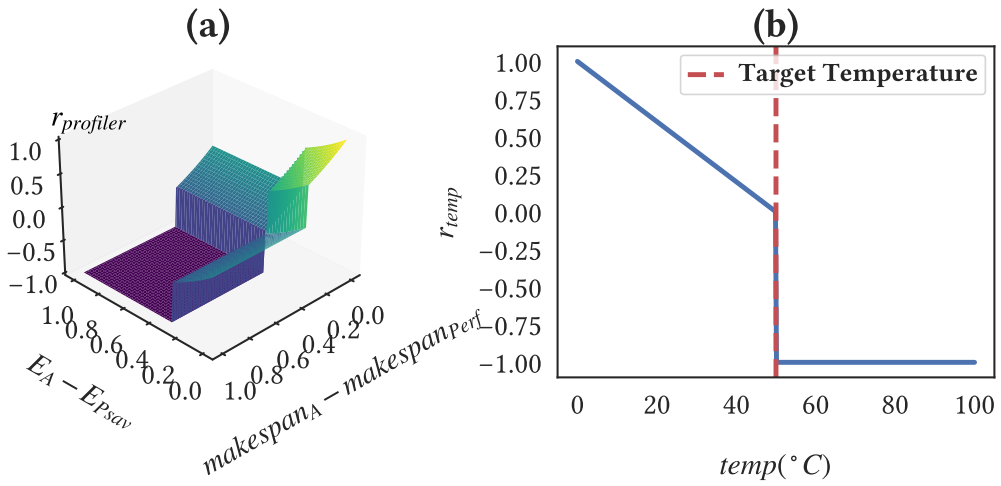
Model-free RL interaction loop

Why Hierarchical Multi-Agent?

- **Action Space Decomposition**
  - Profiler Agent: Selects #cores and base frequency
  - Temperature Agent: Selects core combination based on thermal state
  - Priority Agent: Determines task execution order
- **Collaborative Learning**
  - Joint Action Learners (JAL) coordinate decisions
  - Shared reward signal aligns objectives
- **Scalability**
  - Reduces $O(m^n)$ to $O(m \times n)$
  - Enables scaling to many-core systems

Reward Function Design

- **Multi-Objective Reward**
  - Primary: Makespan reduction
  - Regularizers: Energy, temperature constraints
- **Temporal Shaping**
  - Predicts future rewards for improved sample efficiency
  - Trained reward function estimates cumulative returns



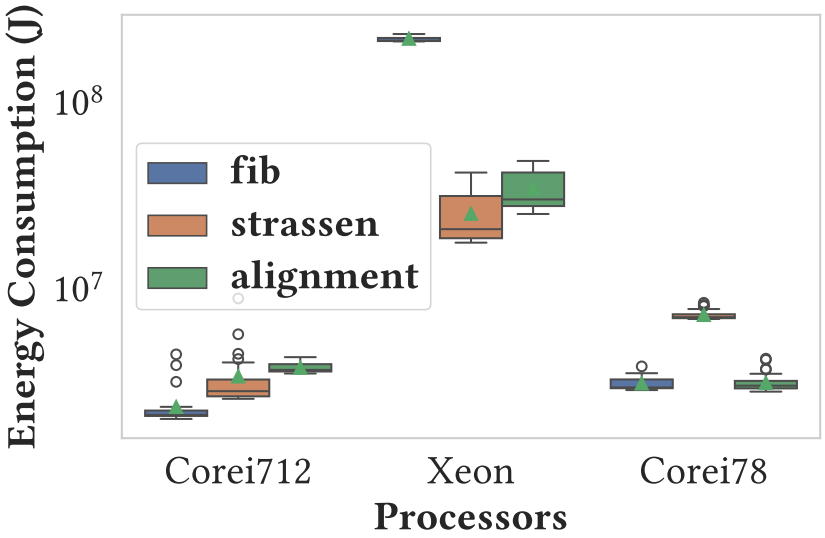
Multi-objective reward: makespan, energy, thermal constraints

Performance Variability Across Platforms

- **Energy Variation**
  - Same benchmark, different platforms
  - Up to 7× energy difference
- **Thermal Behavior**
  - Different cooling capacities
  - Throttling thresholds vary

FFT Benchmark Results:

Platform	Energy (mJ)	Makespan (s)	Temp (°C)
Jetson TX2	9.1	1.13	42.1
Jetson Orin NX	12.4	0.89	38.5
RUBIK Pi	15.8	1.45	44.2



Energy variation across platforms (FFT)

Experimental Platforms

Platform	Cores	Freq (GHz)	TDP* (W)	Architecture
Jetson TX2	6	0.35–2.0	15	Denver2 + A57
Jetson Orin NX	8	0.73–2.2	25	Cortex-A78AE
RUBIK Pi	8	0.4–2.4	10	A76 + A55
Intel Core i7	14	0.8–4.7	125	P-cores + E-cores
Intel Xeon	12	1.0–3.6	150	Server-class

Hardware platforms used for evaluation. *TDP = Thermal Design Power

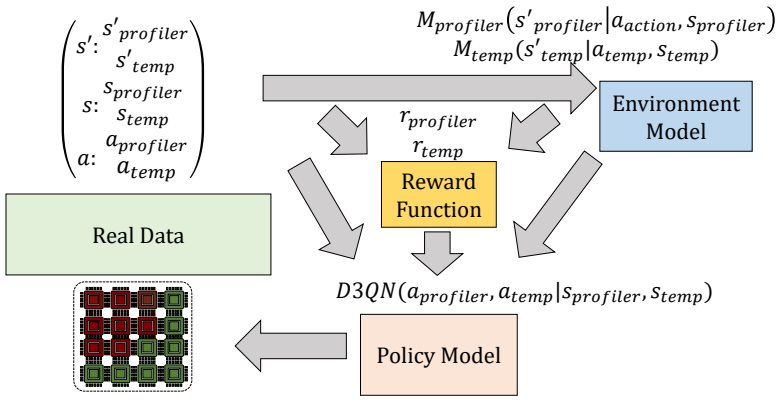
- **Embedded ARM**: Jetson TX2/Orin NX, RUBIK Pi – per-core DVFS, power-constrained
- **Desktop/Server**: Intel Core i7, Xeon – hardware-managed DVFS, performance-oriented
- **Common**: All support thermal monitoring

HiDVFS: Overview

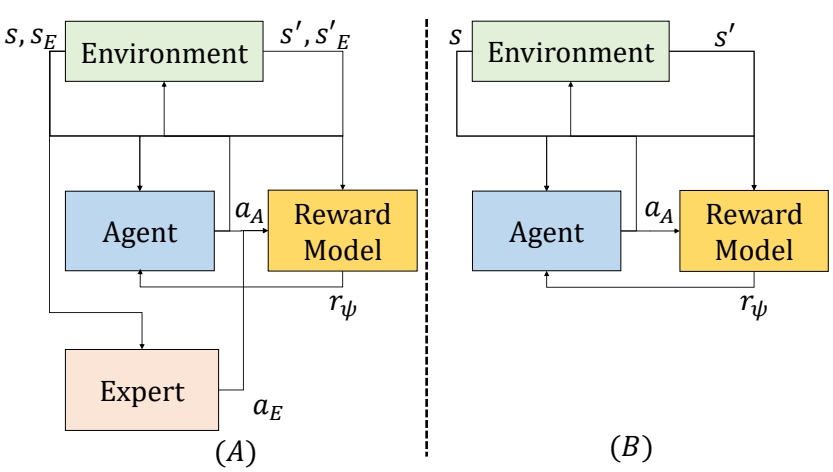
Hierarchical Multi-Agent DVFS Scheduler for OpenMP DAG Workloads

- **Problem**
  - Per-core frequency monitoring lacking in existing DVFS
  - Irregular task patterns not addressed
- **Solution**
  - Three-agent hierarchy: Profiler + Temperature + Priority
  - Double-Deep Dueling Q-Networks (D3QN)
  - Makespan-focused reward with energy/thermal regularizers
- **Key Results**
  - 49.06% makespan reduction
  - 40.95% energy reduction
  - Converges in 12 epochs

HiDVFS: Architecture and Reward Estimation



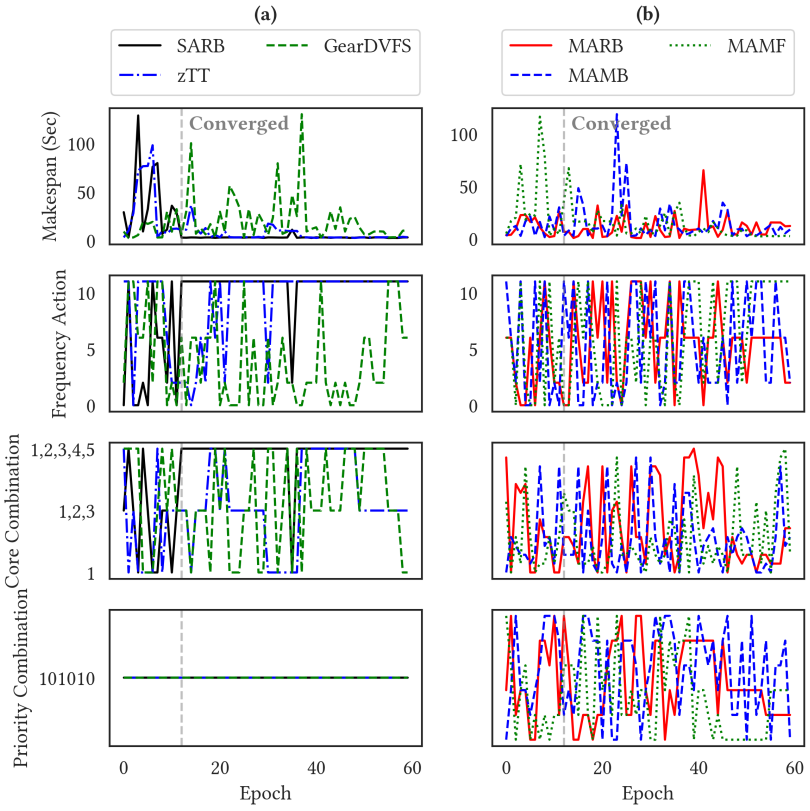
Hierarchical multi-agent architecture



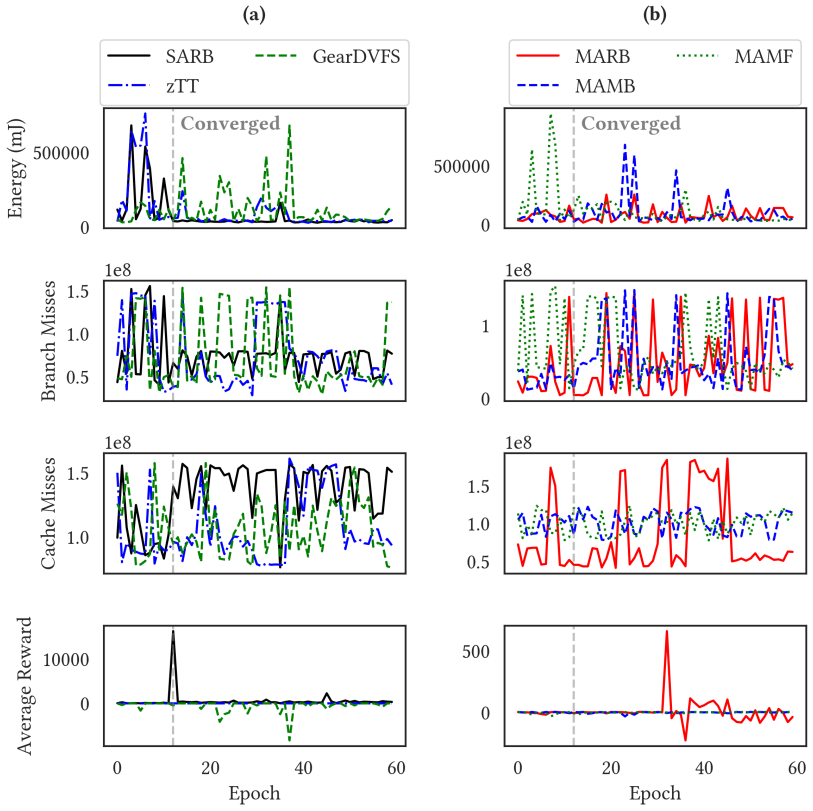
IRL (A) vs. our key-state prediction (B)

- **Profiler Agent**: Selects cores and frequency based on task profile
- **Temperature Agent**: Prioritizes cores based on thermal state
- **Priority Agent**: Resolves resource contention

HiDVFS: Experimental Results



Makespan convergence



Energy convergence

Baselines: zTT [Kim et al. MobiSys'21], GearDVFS [Lin et al. MobiCom'23]. BOTS = Barcelona OpenMP Tasks Suite.

- ☐ Evaluated on NVIDIA Jetson TX2 with FFT benchmark (BOTS)
- ☐ Compared against SARB, MARB, heuristic baselines
- ☐ 89.68% faster convergence than model-free methods

HiDVFS: Performance Comparison

Method	Makespan (s)	Energy (J)	Convergence
Performance Gov.	2.15	0.048	N/A
Ondemand Gov.	2.42	0.041	N/A
SARB (Single-Agent)	1.85	0.038	120 epochs
MARB (Multi-Agent)	1.52	0.032	80 epochs
HiDVFS (Ours)	**1.10**	**0.022**	**12 epochs**

FFT benchmark on Jetson TX2

- **49.06%** makespan reduction vs. multi-agent baseline
- **40.95%** energy reduction vs. multi-agent baseline
- **6.7**$\times$ faster convergence than MARB

HiDVFS: Complete Baseline Comparison (added)

Approach	Energy (J)	Makespan (s)	Branch Miss	Conv. Epochs
Single-Agent Approaches				
zTT	6068.53	705.57	4.4B	15
DynaQ	7960.52	1101.40	4.6B	150
PlanGAN	5404.63	800.72	4.6B	150
GearDVFS	6572.64	1190.95	4.4B	150
HiDVFS_S	**4438.88**	**626.01**	4.4B	**12**
Multi-Agent Approaches				
MAMB	5784.75	861.83	2.9B	150
MAMF	6972.42	962.17	3.8B	44
HiDVFS	**4528.79**	**666.36**	2.8B	84
HiDVFSD3	7665.12	1308.21	2.3B	65

FFT benchmark, summed over 60 epochs on Jetson TX2

Baselines: zTT [Kim et al. MobiSys'21], GearDVFS [Lin et al. MobiCom'23], DynaQ [Angermueller et al. ICLR'19], PlanGAN [Charlesworth & Montana NeurIPS'20]

- HiDVFS_S: 43.17% makespan reduction vs. DynaQ, 44.24% energy reduction
- HiDVFS: 49.06% makespan improvement vs. HiDVFSD3
- HiDVFS: 40.95% energy reduction vs. HiDVFSD3

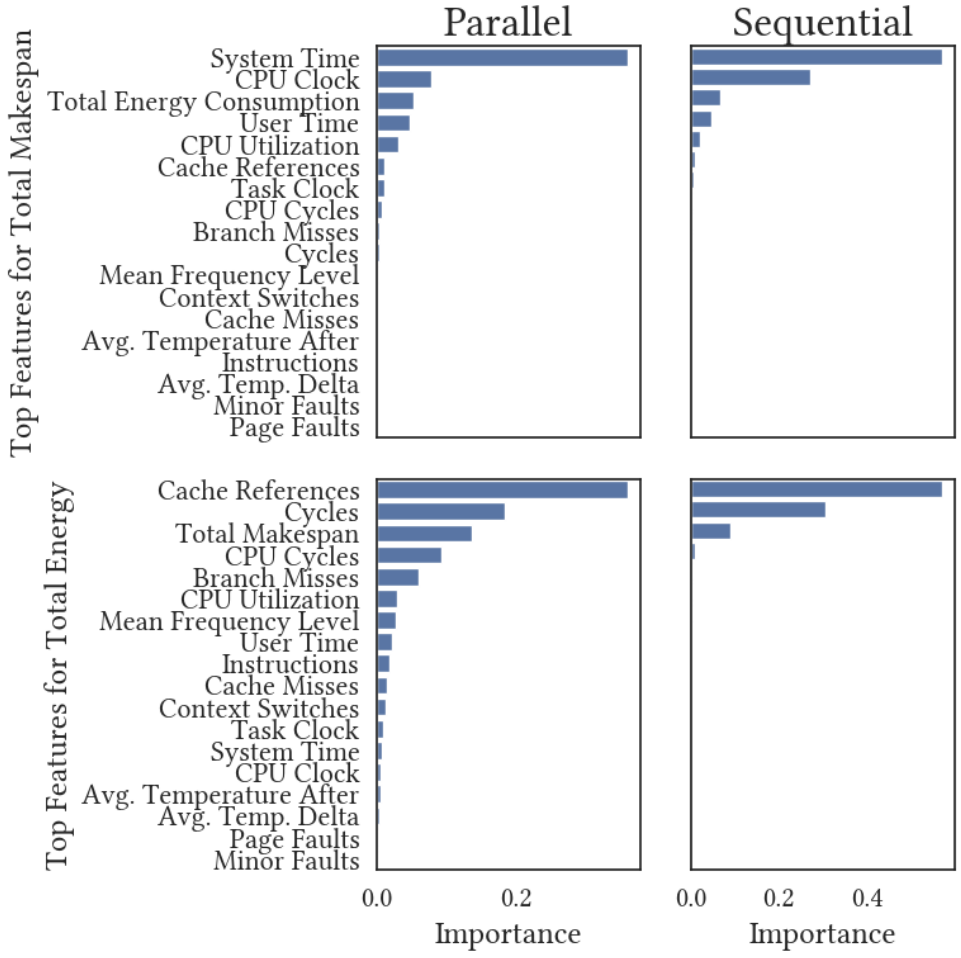
Feature-Aware Allocation: Overview

Statistical Learning for Task-to-Core Allocation

- **Problem**
  - Which features matter for energy-efficient scheduling?
  - Existing approaches ignore parallelism and compute intensity
- **Solution**
  - Statistical feature selection identifies influential features
  - Core type, speed, temperature, application-level parallelism
  - Correlation-aware task allocation
- **Key Results**
  - 10% energy reduction vs. random allocation
  - 5°C temperature reduction
  - 61.6% MSE reduction in thermal prediction

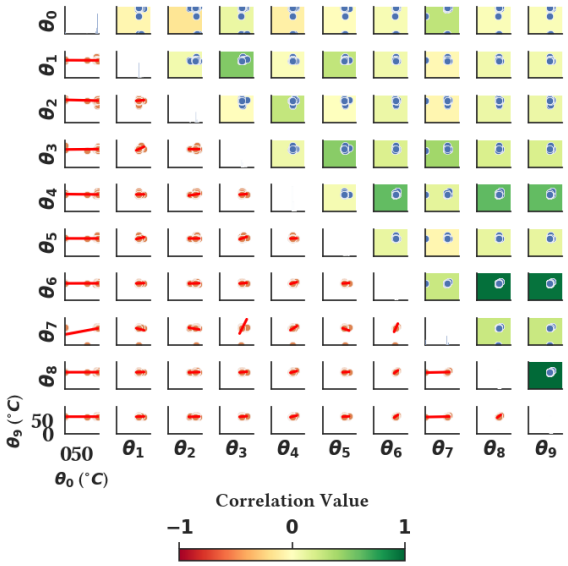
Feature-Aware Allocation: Feature Importance

- **Random Forest Analysis**
  - Identifies critical features
  - Frequency, core type, temperature
  - Application-specific patterns
- **Compressed Models**
  - 16% parameter reduction
  - 6% accuracy improvement
  - Bootstrapped regression

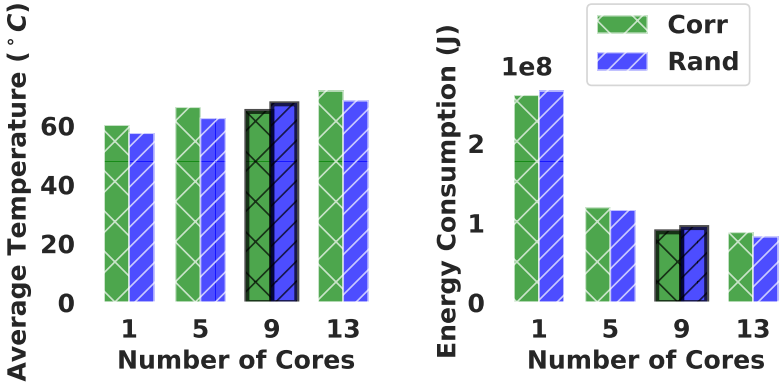


Random Forest feature importance

Feature-Aware Allocation: Thermal Prediction



Core temperature correlations (greener = higher)



- **Correlation-Aware Allocation**
  - Pearson correlation r_{ij} between core temps
  - Core score: $s_i = \frac{1}{m-1} \sum_{j \neq i} |r_{ij}|$
  - Lower score $\rightarrow$ less thermally coupled
- **Task Assignment Strategy**
  - Rank cores by correlation scores
  - Assign tasks to least-correlated cores
- **Benefits**
  - Reduces localized overheating
  - Improves energy efficiency

Feature-Aware: Model Comparison (added)

Model	Temperature		Profiler	
	MSE%	Params	MSE%	Params
FCN	1.0299	2014	3.9047	3787
FCN+RF	0.9808	1694	2.4862	2699
FCN+RF+BS	**0.9640**	**1694**	**2.4669**	**2699**
RNN	1.0119	3070	2.8493	4843
LSTM	1.0307	9310	2.7778	15115
Conv	1.0134	5118	2.8217	6891
Attention	1.0143	6238	2.8933	8011
[Hosseinimotlagh et al. RTSS'21]	2.5000	–	–	–

Intel Core i7 12th Gen (14 cores). FCN=Fully Connected Network, RF=Random Forest, BS=Bootstrapping.

- FCN+RF+BS: **61.4% MSE reduction** vs. SOTA (2.5 → 0.964)
- **16% parameter reduction** with Random Forest feature selection
- Bootstrapping (BS) improves robustness without added parameters

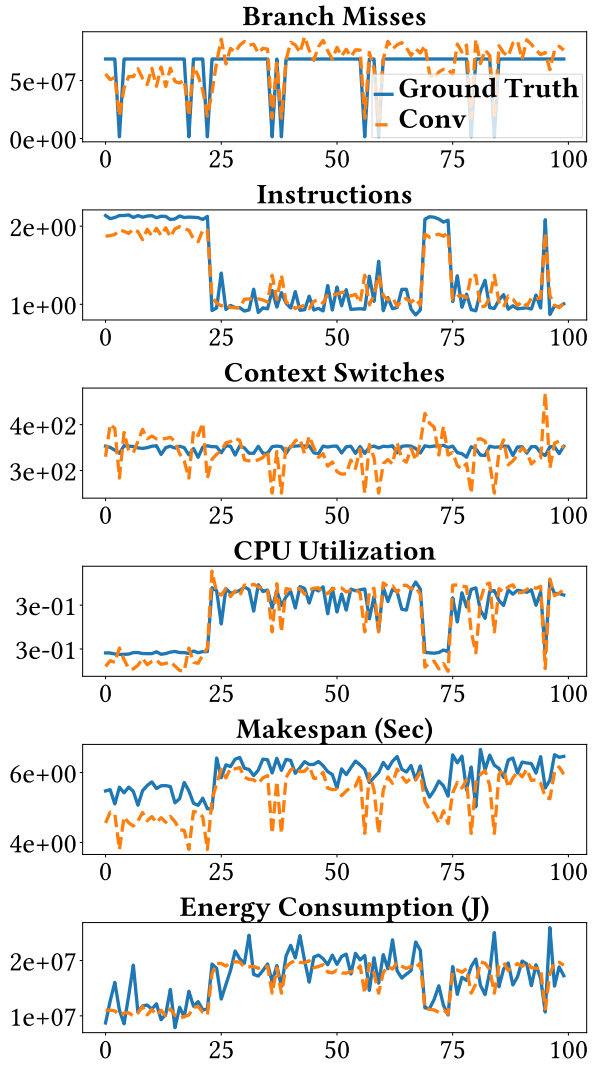
ZeroDVFS: Overview

Model-Based Hierarchical MARL (Multi-Agent RL) for Few-Shot Learning

- **Problem**
  - Offline profiling requires 8–12 hours per platform
  - Cannot adapt at runtime
  - Transfer across platforms fails
- **Solution**
  - Learned environment models for synthetic data generation
  - Dyna-Q inspired framework: Direct RL + model-based planning
  - Two collaborative agents decompose action space
- **Key Results**
  - 7× better temperature prediction
  - Converges in 3 episodes (20× faster)
  - 80,000× faster decision latency (358ms vs. 8–12 hours)

ZeroDVFS: Environment Modeling

- **Profiler Model**
  - Predicts next state from (state, action)
  - LSTM (Long Short-Term Memory)-based for temporal dynamics
- **Temperature Model**
  - Regression for thermal prediction
  - 7× better accuracy than prior work
- **Synthetic Data**
  - Generate rollouts from models
  - Enables few-shot learning



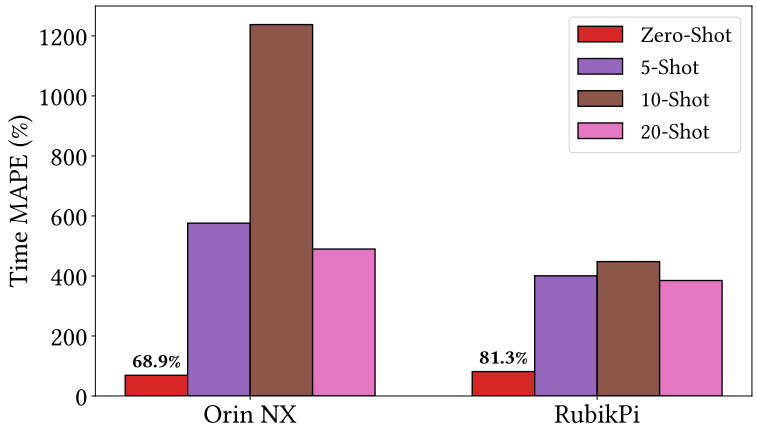
Profiler model prediction accuracy

ZeroDVFS: Cross-Platform Transfer

Energy Efficiency (TX2)

Method	Energy	Makespan
Heuristic	64.5 mJ	4.52 s
zTT	31.2 mJ	1.93 s
ZeroDVFS	**9.1 mJ**	**1.13 s**

zTT [Kim et al. MobiSys'21]



(a) Transfer learning comparison

Transfer Learning (MAPE: Mean Absolute Percentage Error)

Transfer Path	Zero-Shot
TX2 (Source)	8.6%
TX2 $\rightarrow$ Orin NX	68.9%
TX2 $\rightarrow$ RubikPi	81.3%

$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y - \hat{y}}{y} \right| \times 100\%$ (scale-independent)

Why MAPE?

- Scale-independent: comparable across platforms
- Interpretable: % error in time prediction

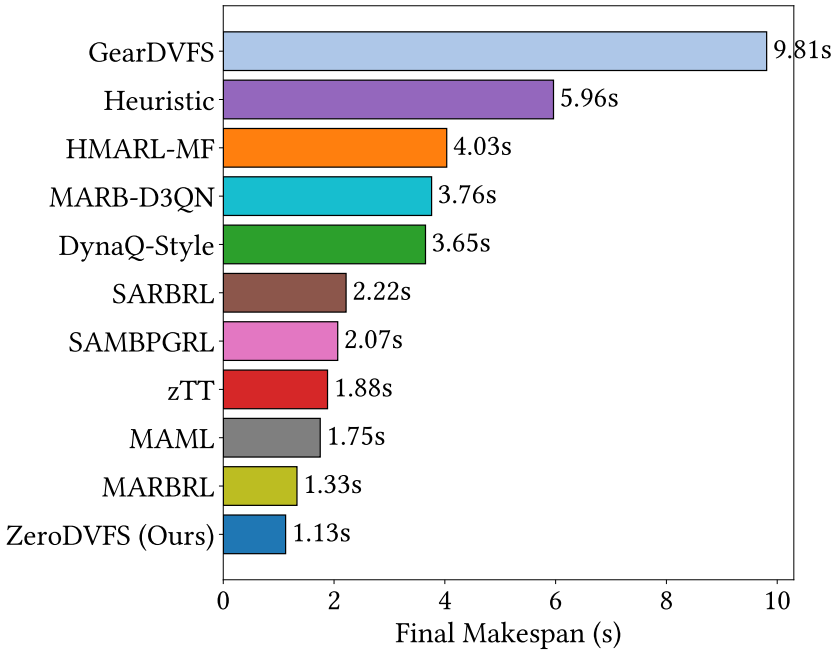
Advantage of Zero-Shot Transfer:

- One-time training: $\sim$30 min (done once)
- Runtime decision: 358ms (vs 8-12h table regen)
- Fine-tuning optional for improved accuracy

ZeroDVFS: Performance Ranking

- Rankings computed from mean makespan over final 10 episodes of 100-episode training
- ZeroDVFS outperforms MAML (1.75s), zTT (1.88s), GearDVFS (9.81s)
- ZeroDVFS achieves best performance at 1.13s

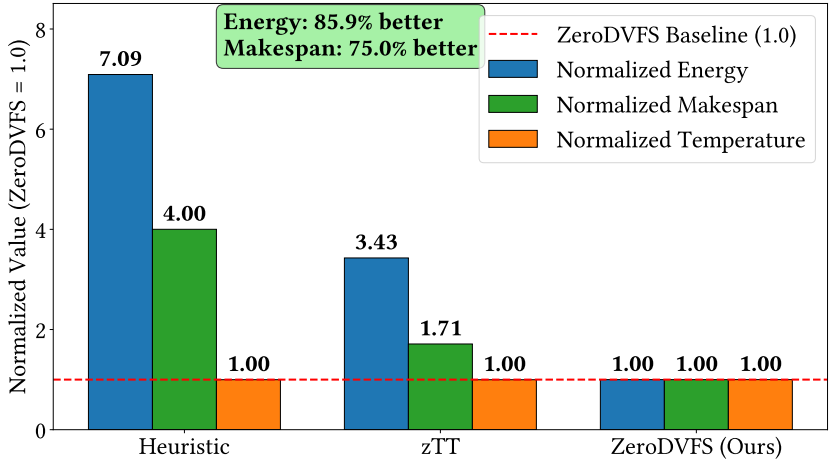
Baselines:
zTT [Kim et al. MobiSys'21]
GearDVFS [Lin et al. MobiCom'23]
MAML [Finn et al. ICML'17]



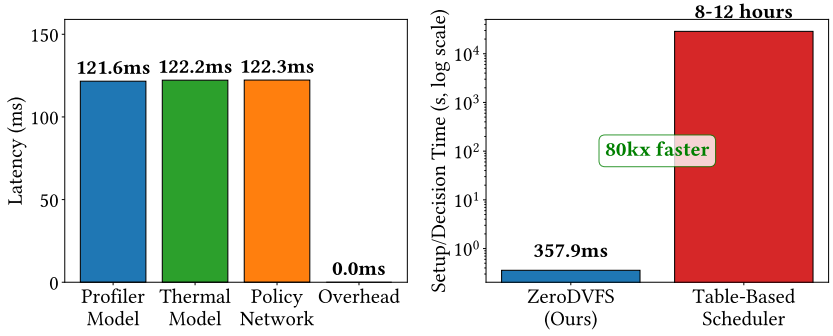
Performance ranking on BOTS FFT (Jetson TX2)

ZeroDVFS: Decision Latency Analysis

- **Latency Breakdown (from paper)**
  - Profiler Agent: 121.6ms ± 48.3ms
  - Thermal Agent: 122.2ms ± 44.5ms
  - Policy Execution: 122.3ms ± 46.6ms
  - **Total: 357.9ms ± 97.1ms**
- **Comparison**
  - Table-based: 8–12 hours
  - Model-free RL: Minutes × 400+
  - ZeroDVFS: **358ms**
- **Impact**
  - 80,000× faster than tables
  - Enables runtime adaptation



(a) Energy efficiency comparison



(b) Decision latency breakdown on TX2

Baselines: zTT [Kim et al. MobiSys'21], GearDVFS [Lin et al. MobiCom'23]

ZeroDVFS: 80,000× Speedup Derivation

- **Table-Based Profiling Time**:

$$\boxed{T_{\text{table}} = m \times k \times |\Gamma| \times \rho \times \bar{t}}$$

Parameters:

- m = number of cores (6 for TX2)
- k = frequency levels (12 for TX2)
- $|\Gamma|$ = task set cardinality
- ρ = repetitions per config
- $\bar{t}$ = avg execution time

Comparison:

- Table-based: 8–12 hours
- ZeroDVFS: **358ms**

Speedup Calculation:

- $\frac{10 \text{ hours}}{358 \text{ ms}} = \frac{36{,}000{,}000 \text{ ms}}{358 \text{ ms}}$
- $\approx$ **100,000**× faster

ZeroDVFS enables runtime adaptation impossible with tables

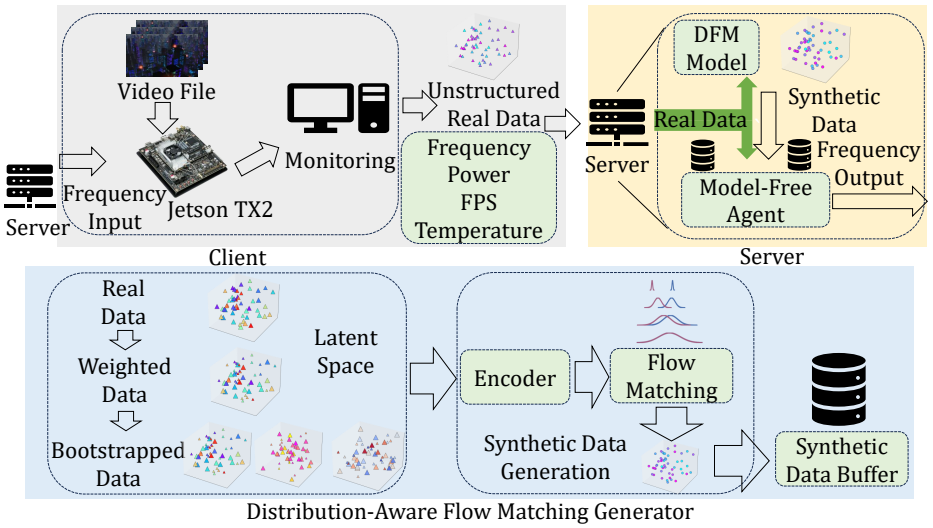
FlowRL: Overview

Flow Matching for Few-Shot Reinforcement Learning

- **Problem**
  - Generating realistic unstructured sensor data is challenging
  - Traditional RL relies on extensive datasets
  - Overfitting and data correlation in model-based RL
- **Solution**
  - Distribution-aware flow matching for synthetic data
  - Bootstrapping improves latent space robustness
  - Random Forest feature weighting for precision
- **Key Results**
  - Stable Q-value convergence
  - 30% frame rate improvement in early training
  - Law of Large Numbers convergence guarantees

FlowRL: Algorithmic Overview

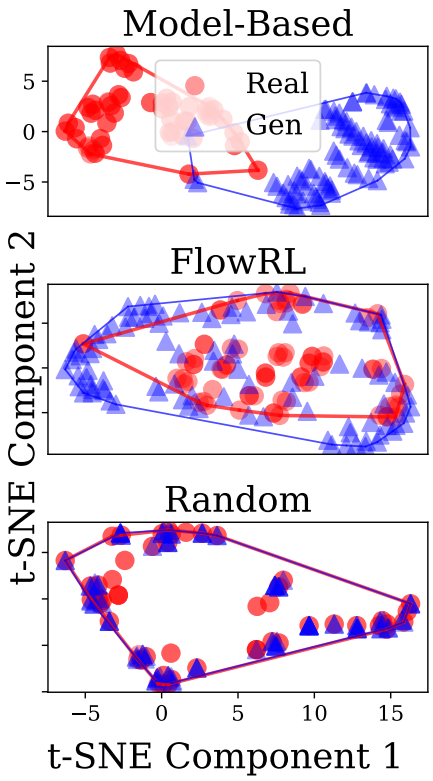
- Steps for Flow Matching in RL:
  - Initialize real/synthetic data buffers, DQN, flow matching model.
  - Collect real trajectories and store.
  - Train flow matching model when sufficient data is collected.
  - Generate synthetic trajectories via resampling and flow matching.
  - Update DQN with combined real/synthetic data.
  - Repeat for multiple iterations.



Workflow of the proposed distribution-aware flow matching RL approach.

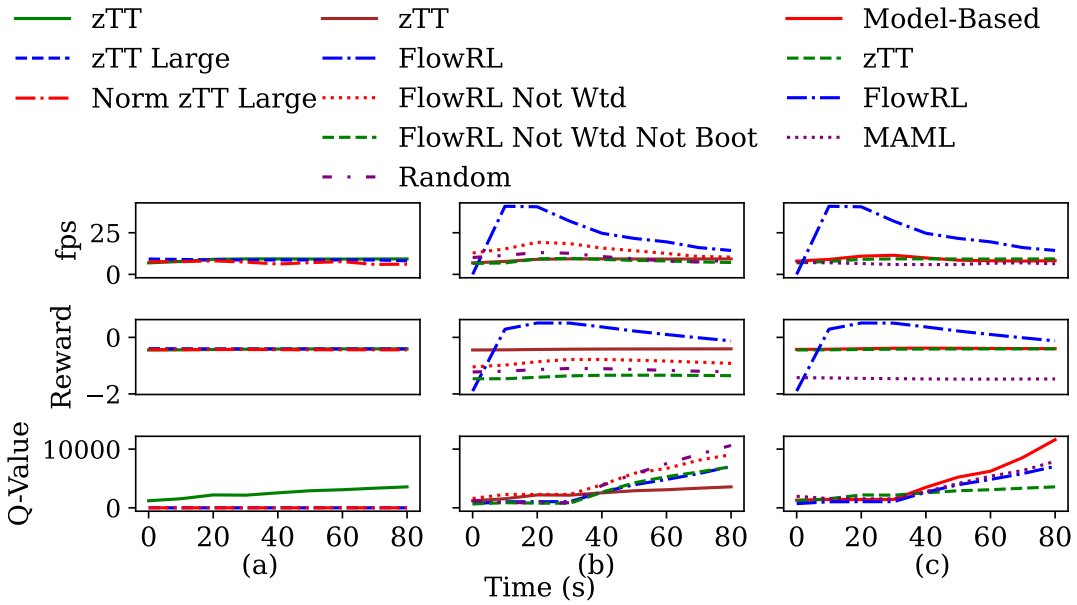
FlowRL: Convergence and Results

- **Q-Value Convergence**
  - Stable max Q-value with flow-augmented data
  - Law of Large Numbers guarantees
  - Avoids overfitting in unstructured data
- **Key Results**
  - Up to 35% higher frame rates vs baselines
  - Faster Q-value convergence
  - DTW (Dynamic Time Warping): FlowRL (7,341) vs MB (10,640)
- **Comparison with Baselines**
  - Outperforms DQN, DDQN, MAML
  - Better sample efficiency
  - Lower variance in generated samples



Real vs Generated data t-SNE (t-distributed Stochastic Neighbor Embedding). FlowRL covers real data and generalizes beyond

FlowRL: Performance Comparison (added)



Performance comparisons across methods (FPS, Reward, Q-value over 90s). BOTS = Barcelona OpenMP Tasks Suite.

- **Left**: Model-free variants (zTT, extended features, normalized) show similar stable FPS ≈10.
- **Middle**: FlowRL (weighted+bootstrapped) achieves faster convergence vs FM variants and random.
- **Right**: FlowRL outperforms MAML, zTT, and model-based approaches in all metrics.

Baselines: zTT [Kim et al. MobiSys'21], MAML [Finn et al. ICML'17]

GraphPerf-RT: Overview

Graph-Driven Performance Prediction with Uncertainty

- **Problem**
  - Classical heuristics struggle with workload irregularity
  - Tabular regressors discard structural information
  - Model-free RL risks overheating
- **Solution**
  - Heterogeneous graph: DAG topology + CFG (Control Flow Graph) semantics + runtime context
  - Multi-task evidential heads with calibrated uncertainty
  - Graph Attention Networks for representation learning
- **Key Results**
  - $R^2 > 0.95$ with ECE < 0.05
  - 66% makespan reduction, 82% energy reduction
  - Validated on TX2, Orin NX, RUBIK Pi

GraphPerf-RT: Heterogeneous Graph Representation

- **Node Types**
  - Task nodes: DAG structure, precedence constraints
  - CFG nodes: Control flow, basic blocks
  - Core nodes: DVFS state, thermal readings
- **Edge Types**
  - Precedence edges: Task dependencies
  - Placement edges: Task-to-core assignment
  - Contention edges: Resource sharing
- **Predictions**
  - Makespan, energy, cache misses, branch misses, utilization
  - Calibrated uncertainty for risk-aware scheduling

GraphPerf-RT: Evidential Deep Learning

- **Key Idea**: Instead of predicting just a value, predict a *distribution over predictions*
  - Standard NN: predicts $\hat{y}$ (point estimate, no confidence)
  - Ensemble: runs N models, expensive ($N\times$ compute)
  - **Evidential**: outputs 4 params $\rightarrow$ full uncertainty in **1 forward pass**
- **How It Works**: Network outputs $(\gamma, \nu, \alpha, \beta)$
  - γ: predicted mean (the actual prediction)
  - ν: "virtual sample count" (confidence in prediction)
  - α, β: control spread of uncertainty
- **Two Types of Uncertainty Separated**:
  - **Aleatoric**: inherent noise (e.g., sensor measurement noise)
  - **Epistemic**: model doesn't know (unseen configurations)
- **Why It Matters**: Filter unsafe predictions before executing
  - ECE < 0.05: when model says 90% confident, it's right $\sim$90%

GraphPerf-RT: Ablation Study

Configuration	R^2	MAE	ECE
MLP Baseline	0.78	0.142	0.18
GCN (no attention)	0.85	0.098	0.12
GAT (single head)	0.91	0.067	0.08
GAT (no evidential)	0.94	0.045	N/A
GraphPerf-RT (full)	**0.96**	**0.038**	**0.04**

BOTS benchmarks on Jetson TX2. MLP=Multi-Layer Perceptron, GCN=Graph Convolutional Network, GAT=Graph Attention Network.

- Multi-head attention improves feature weighting
- Heterogeneous edges capture task-core interactions
- Evidential heads add calibrated uncertainty at minimal cost

GraphPerf-RT: RL Integration Analysis

Method	Type	Agent	Makespan (s)	Energy (J)
SAMFRL	Model-Free	Single	2.85 ± 1.66	0.033 ± 0.026
SAMBRL	Model-Based	Single	3.56 ± 3.07	0.038 ± 0.032
MAMBRL-D3QN	**Model-Based**	**Multi**	$\mathbf{0.97 \pm 0.35}$	$\mathbf{0.006 \pm 0.005}$

200 episodes, 5 seeds (42, 123, 456, 789, 1024) on Jetson TX2

- **Key Findings**
  - **66% makespan reduction**: SAMFRL 2.85s → MAMBRL-D3QN 0.97s
  - **82% energy reduction**: SAMFRL 0.033J → MAMBRL-D3QN 0.006J
- **Why Multi-Agent + Model-Based Works**
  - Per-core DVFS decisions coordinated via shared world model
  - GraphPerf-RT provides accurate predictions for Dyna-style planning
  - Uncertainty gating reduces unsafe scheduling proposals (18% → 3%)

Limitations and Scope (added)

- **Programming Model Constraints**
  - Framework designed for OpenMP task-based workloads
  - Extension to CUDA, OpenCL requires additional work
- **Platform Dependencies**
  - Requires userspace DVFS control (cpufreq userspace governor)
  - API (Application Programming Interface) and thermal zone naming varies across boards
  - Cross-platform transfer shows 18% RMSE degradation
- **Thermal Dynamics**
  - Steady-state thermal modeling; transients under long runs not fully captured
  - Static DAG assumption; dynamic task creation not handled
- **Evaluation Scope**
  - Validated on BOTS + PolyBench (42 benchmarks)
  - Real-world complex applications need further study

Summary of Contributions

- **HiDVFS**: Hierarchical multi-agent RL reduces action space exponentially
- **Feature-Aware Allocation**: Statistical learning identifies critical scheduling features
- **ZeroDVFS**: Model-based RL enables few-shot learning and cross-platform transfer
- **FlowRL**: Flow matching generates synthetic data for sample-efficient RL
- **GraphPerf-RT**: Graph-based prediction with calibrated uncertainty enables risk-aware scheduling

Overall Impact: Data-efficient AI methods enable practical deployment of RL-based scheduling on resource-constrained embedded systems

Publications

Submitted (arXiv available):

1. **HiDVFS**: M. Pivezhandi, A. Saifullah. "HiDVFS: Hierarchical Multi-Agent DVFS Scheduler for OpenMP DAG Workloads." *IPDPS 2026* (submitted)
2. **GraphPerf-RT**: M. Pivezhandi, M. Banisharif, S. Bakhshan, A. Saifullah, A. Jannesari. "GraphPerf-RT: Graph-Driven Performance Prediction with Evidential Uncertainty." *AISTATS 2026* (submitted, arXiv available)
3. **ZeroDVFS**: M. Pivezhandi, M. Banisharif Dehkordi, A. Saifullah. "ZeroDVFS: Zero-Shot Model-Based Core and Frequency Allocation." *ECAI 2026* (submitted, arXiv available)
4. **FlowRL**: M. Pivezhandi, A. Saifullah. "FlowRL: Flow-Augmented Few-Shot RL for Semi-Structured Sensor Data." *ICLR 2026* (submitted, arXiv available)

Published:

5. M. Pivezhandi, V. P. Modekurthy, A. Saifullah. "Feature-Aware Task-to-Core Allocation via Statistical Learning." *RTCSA 2024*
6. M. Pivezhandi, A. Jain, A. Saifullah, A. Jannesari. "Work-In-Progress: Energy and Thermal-Aware Scheduling based on HMARL for OpenMP DAG Workloads." *RTSS 2024*

Other Work (Beyond Dissertation)

- **M. Pivezhandi**; FPGA-Based Vision Accelerators Survey (single author, to be submitted)
- S. Fahimda, **M. Pivezhandi**, A. Saifullah, N. Fisher; Global EDF Scheduling of Parallel Tasks on Uniform Multicores, Submitted to IPDPS26
- A. Ahmed, **M. Pivezhandi**, Z. Guo, J. Li, V. P. Modekurthy, A. Saifullah; Precise Scheduling of DAG Tasks with Dynamic Power Management, *ECRTS'23*
- Q. Bashir, **M. Pivezhandi**, A. Saifullah; Energy- and Temperature-aware Scheduling: From Theory to Implementation on Intel Processor, *HPCC'22*
- V. Bengre, M. R. HoseinyFarahabady, **M. Pivezhandi**, A. Y. Zomaya, A. Jannesari; A Learning-based Scheduler for High Volume Processing in Data Warehouse using GNNs, *ICPDC'21*
- **M. Pivezhandi**, P. H. Jones, J. Zambreno; Parahist: FPGA Implementation of Parallel Event-based Histogram for Optical Flow, *ASAP'20*
- **M. Pivezhandi**, M. Eshghi; ASIP Design for Two Dimensional CORDIC Based DFT and DCT Algorithms, *ICCKE'16*

Future Work

- **Extending HiDVFS & Feature-Aware Allocation**
  - RISC-V processors and custom accelerators
  - GPU-CPU heterogeneous systems
- **Extending ZeroDVFS & FlowRL**
  - Meta-learning for rapid platform adaptation
  - Online continual learning for evolving workloads
  - Distributed multi-node clusters
- **Extending GraphPerf-RT**
  - Foundation models for code-to-performance prediction
  - Hard real-time guarantees with uncertainty-aware scheduling
  - Safety-constrained reinforcement learning

Key Takeaways

1. **Hierarchy enables scalability**: Multi-agent decomposition reduces $O(m^n) \rightarrow O(m \times n)$
2. **Features matter**: Statistical feature selection improves energy and thermal efficiency
3. **Models enable transfer**: Environment modeling allows zero-shot cross-platform deployment
4. **Distribution-aware augmentation**: Flow matching generates realistic synthetic data
5. **Graphs capture structure**: Heterogeneous graphs unify DAG topology with runtime context
6. **Uncertainty enables safety**: Evidential learning provides calibrated confidence for risk-aware decisions

Thank You!

Questions?

Mohammad Pivezhandi
pivezhandi@wayne.edu

WAYNE STATE UNIVERSITY

Backup Slides

Mathematical Formulations

Backup: Limitations of Existing Solutions

- **Hardware Solutions (DVFS)**
  - Lack per-core frequency tracking
  - Cluster-level scaling ignores individual core temperatures
  - Cannot adapt to irregular workload patterns
- **Software Solutions (Schedulers)**
  - Ignore runtime features: branch misses, cache behavior
  - Temperature-agnostic task allocation
  - Static policies don't generalize
- **Learning-Based Solutions**
  - Sample inefficient: thousands of episodes
  - High computational overhead
  - Platform-specific, no transfer learning

Backup: Reward Function Formulation (added)

- **Variables**: M: Makespan (s), E: Energy (J), T: Temperature (°C), $\beta \in [0, 1]$: weight
- **Profiler Agent Reward**:

$$R_1 = \beta \cdot \frac{M_{\text{target}}}{M + \epsilon} + (1 - \beta) \cdot \frac{E_{\text{target}}}{E + \epsilon}$$

- **Thermal Agent Reward**:

$$R_2 = 1 - 0.05 \cdot |T - T_{\text{target}}|$$

- **Priority Agent Reward**:

$$R_3 = \frac{M_{\text{target}} - M}{M_{\text{target}}}$$

Higher reward for lower makespan/energy, thermal stability near target

Backup: Flow Matching Objective (added)

- **Flow Matching Optimization**:
  - $v_{\mathcal{W}}(x_t)$: Velocity field for data x_t with parameters $\mathcal{W}$
  - B: Number of samples, $p_t^b(x)$: Data distribution
  - $w(x_t^b)$: Target velocity for sample b

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}_{\substack{t\in[0,1] \\ x_t^b \sim p_t^b(x)}} \left[\| v_{\mathcal{W}}(x_t^b) - w(x_t^b) \|^2 \right]$$

- **Conditional Flow Matching with Feature Weights**:
  - $\lambda_i = \frac{\text{Importance}(x_{t,i})}{\sum_{j=1}^{d} \text{Importance}(x_{t,j})}$: Weight from Random Forest

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}_{\substack{t\in[0,1] \\ x_t^b \sim p_t^b(x) \\ z\sim q(z)}} \left[\sum_{i=1}^{d} \lambda_i \| v_{\mathcal{W}}(x_{t,i}^b; z) - w(x_{t,i}^b; z) \|^2 \right]$$

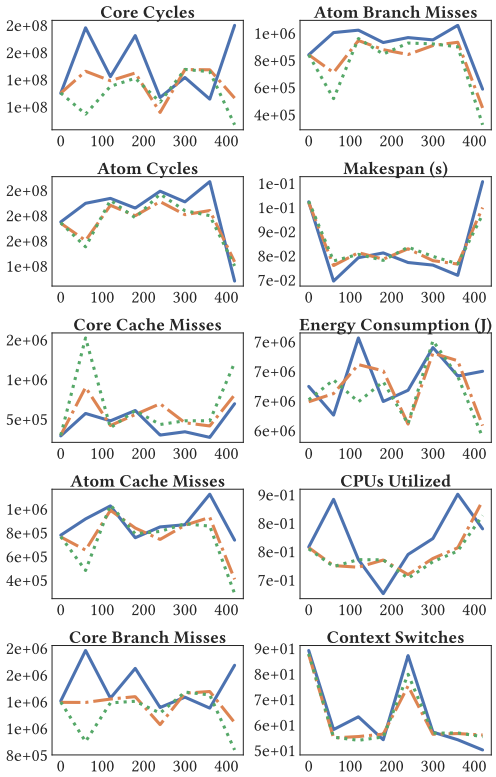
Backup: Pearson Correlation for Feature Selection (added)

- **Pearson Correlation Coefficient**:
  - Analyzes pairwise relationships between features
  - $\text{corr}_{ij} \in [-1, 1]$: 1 = positive, -1 = negative, 0 = none

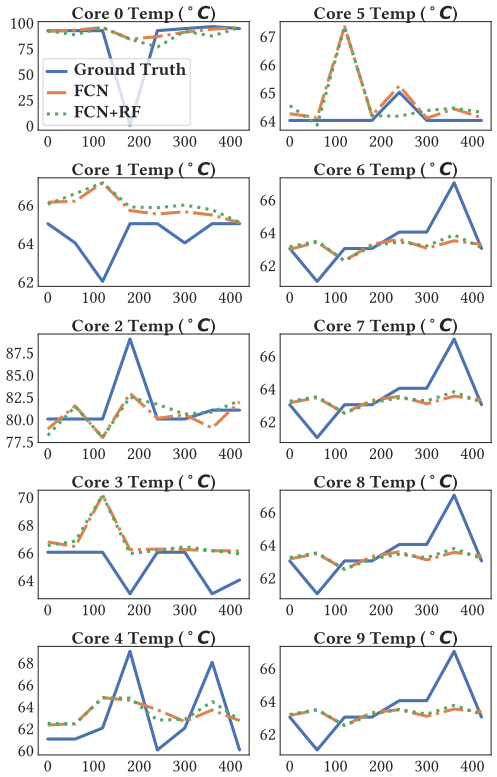
$$\text{corr}_{ij} = \frac{\sum_{k=1}^{n}(x_{i,k} - \bar{x}_i)(x_{j,k} - \bar{x}_j)}{\sqrt{\sum_{k=1}^{n}(x_{i,k} - \bar{x}_i)^2}\sqrt{\sum_{k=1}^{n}(x_{j,k} - \bar{x}_j)^2}}$$

- **Temperature-Based Core Prioritization**:
  - Compute correlation r_{ij} between core temperatures
  - Assign tasks to least-correlated (coolest) cores
  - Reduces thermal hotspots and improves energy efficiency

Backup: Feature-Aware Allocation Results (added)



Profiler agent



Temperature agent

- Evaluated on Intel Core i7 12th Gen (14 cores)
- Validated across diverse hardware platforms
- State-of-the-art Linux governors as baselines

Backup: Regret Bounds and Model Discrepancy (added)

- **Model Discrepancy Bound**:
  - $\widehat{\mathcal{M}}$: Learned environment model
  - $\mathcal{M}^*$: True environment dynamics
  - Synthetic data reduces discrepancy: $\|\widehat{\mathcal{M}} - \mathcal{M}^*\| \leq \varepsilon$
- **Sublinear Regret**:

$$\text{Reg}(T) = O(\sqrt{T}) \quad \text{if } \varepsilon \text{ is small}$$

  - Regret: difference between agent and optimal policy
  - Average regret: $\frac{\text{Reg}(T)}{T} \to 0$ as $T \to \infty$
  - $T = T_R + T_S$ (real + synthetic samples)
- **Key Insight**: Flow matching improves coverage of state-action pairs

Backup: Dueling Double DQN (D3QN) (added)

- **Q-Value Decomposition**: $V(s)$ state value, $A(s,a)$ advantage

$$Q(s,a;\theta) = V(s;\theta_V) + \left(A(s,a;\theta_A) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a';\theta_A) \right)$$

- **Double DQN Target**: Action via θ, value via θ^-

$$y = r + \gamma \cdot Q(s', \arg\max_{a'} Q(s',a';\theta);\theta^-)$$

- **Bellman Loss**:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(y - Q(s,a;\theta))^2 \right]$$

Reduces overestimation bias; separates state value from action advantages

Backup: Evidential Deep Learning Explained (added)

The Problem: Standard NNs give point predictions without knowing "how sure" they are.
The Solution: Train the network to output parameters of a probability distribution.

- **Normal-Inverse-Gamma (NIG) Prior**: Network outputs 4 values:
  - γ: the prediction itself (e.g., predicted makespan = 1.2s)
  - ν: "virtual evidence" — higher ν = more confident
  - α, β: shape parameters controlling uncertainty spread
- **Intuition**: Think of ν as "how many similar examples the model has seen"
  - High ν: model has seen many similar inputs $\rightarrow$ confident
  - Low ν: new/unusual input $\rightarrow$ uncertain
- **Uncertainty Formulas**:
  - Aleatoric (data noise): $\frac{\beta}{\alpha-1}$ — irreducible, from noisy sensors
  - Epistemic (model uncertainty): $\frac{\beta}{\nu(\alpha-1)}$ — reducible with more data
- **Safety Gating**: Only act when epistemic uncertainty $\leq$ threshold η

Backup: Action Space Complexity (added)

- **Problem: Exponential Action Space**
  - n cores, m frequency levels
  - Joint action space: $O(m^n)$ combinations
  - Jetson TX2: $12^6 \approx 2.98$ million combinations
- **Solution: Hierarchical Decomposition**
  - Profiler Agent: selects $(n_{\text{cores}}, f_{\text{base}})$
  - Thermal Agent: prioritizes cores by temperature clusters
  - Priority Agent: adjusts task priorities
- **Reduced Complexity**:

$$O(m^n) \rightarrow O(m \times n)$$

  - Example: $12 \times 6 = 72$ actions (vs. 2.98 million)
  - **41,389**$\times$ **reduction** in action space

Backup: Reward Definition with Model (added)

- M: Makespan (s), E: Energy (J), T: Temperature (°C)
- $\beta \in [0, 1]$: Weight, ϵ: small constant
- R_1: Profiler reward

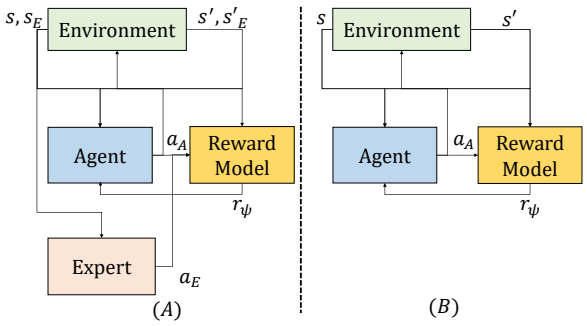
$$R_1 = \beta \frac{M_{\text{target}}}{M + \epsilon} + (1 - \beta) \frac{E_{\text{target}}}{E + \epsilon}$$

- R_2: Thermal reward

$$R_2 = 1 - 0.05 |T - T_{\text{target}}|$$

- R_3: Priority reward

$$R_3 = \frac{M_{\text{target}} - M}{M_{\text{target}}}$$



Reward Estimation Model

Backup: Feature Importance Table (added)

Dataset	Random Forest (Top Features)	Stepwise OLS (Top Features)
Profiler	Task ID, Actions, Temp Core 0-13, ΔTemp Core 0-13, Avg Temp, CPU Cycles, Cache Refs, Misses, Branch Misses, Page Faults, Context Switches	Task ID, Temp Core 1, Temp Core 13
Temperature	Task ID, Actions, Temp Core 0-9, 11-13, ΔTemp Core 0-13, Avg Temp	Task ID, Temp Core 0, 2-4, 7, 10, Avg Temp

- **Insight**: Task ID and core temperatures (Core 1, 13) are key for energy prediction
- Random Forest captures non-linear relationships
- Stepwise OLS identifies minimal sufficient features

Backup: Distribution-Aware Flow Matching (added)

- **Problem**: Limited data hinders few-shot RL for DVFS
- **Solution**: Generates synthetic data using flow matching
- Optimizes velocity field:
  - $v_{\mathcal{W}}(x_t)$: Velocity field for data x_t with parameters $\mathcal{W}$
  - B: Number of samples, $p_t^b(x)$: Data distribution
  - $w(x_t^b)$: Target velocity for sample b

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}_{\substack{t \in [0,1] \\ x_t^b \sim p_t^b(x)}} \left[\| v_{\mathcal{W}}(x_t^b) - w(x_t^b) \|^2 \right]$$

Flow matching learns continuous transformations from noise to data distribution

Backup: FlowRL Flow Matching Architecture (added)

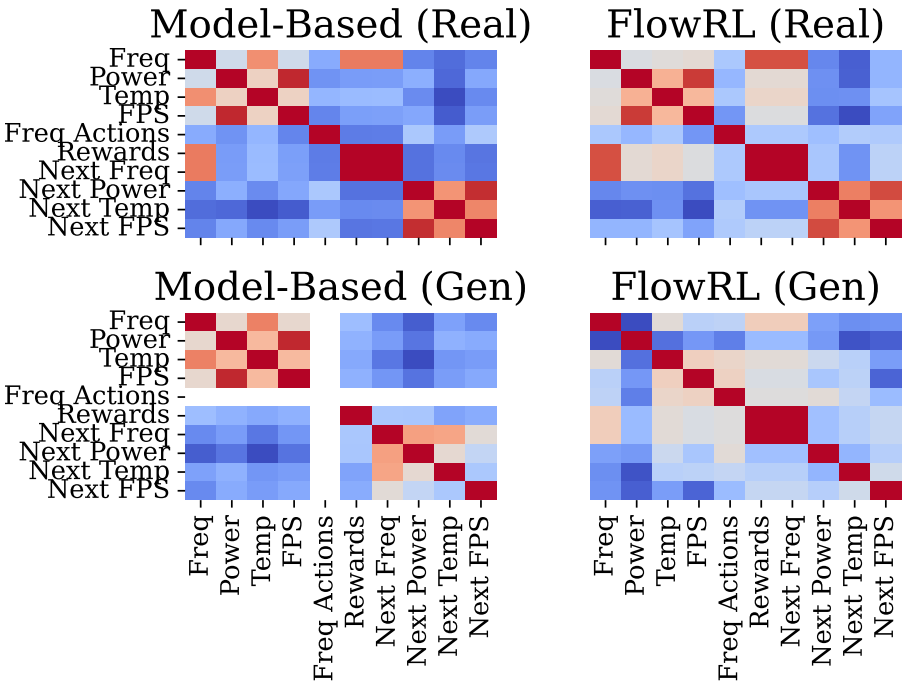
- **Flow Matching**
  - Exact likelihood estimation (advantage over diffusion)
  - Sample-efficient generative modeling
- **Statistical Learning Integration**
  - Bootstrapping for robustness
  - Feature weighting via Random Forests
  - Distribution-aware augmentation
- **DVFS Application**
  - Generate synthetic sensor data
  - Augment real experience buffer
  - Few-shot RL for resource-constrained systems

Backup: FlowRL Conditional Objective and Correlation (added)

- **Conditional flow matching with feature weights**:
  - λ_i: Weight for feature i
  - z: Context variable (e.g., workload)
  - d: Number of features

$$\min_{\mathcal{W}} \frac{1}{B} \sum_{b=1}^{B} \mathbb{E}\left[\sum_{i=1}^{d} \lambda_i \|v_{\mathcal{W}} - w\|^2\right]$$

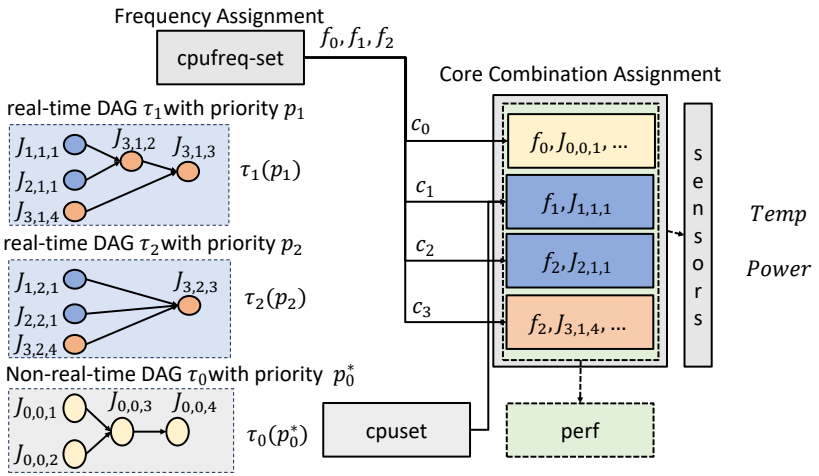
- **Correlation Analysis**:
  - **FlowRL**: Mirrors real correlations
  - **Pure FM**: Dilutes relationships
  - **Model-Based**: Overfits patterns



Correlation matrix comparisons across methods.

Backup: Task Model for Parallel DAG Workloads (added)

- Consider n aperiodic parallel tasks $\tau_1, \tau_2, \ldots, \tau_n$ on m heterogeneous cores
- Each task τ_i is a DAG: $\tau_i = \{\tau_{i,1}, \tau_{i,2}, \ldots, \tau_{i,j}\}$
  - Nodes = subtasks, Edges = dependencies
- Execution instances are jobs $J_{k,i,j}$ (core k, task i, subtask j)
- Priority $P_i \in [1, 99]$ determines scheduling precedence
- Example: Jobs $J_{1,i,1}$ and $J_{2,i,1}$ on cores c_1, c_2 precede $J_{3,i,2}$ on c_3



DAG task model showing subtask dependencies and core allocation

Backup: Why RL-Based Scheduler? (added)

- Temperature-aware task-to-core allocation to reduce makespan and energy.
- Hierarchical MARL to manage complexity in scheduling DAG workloads.

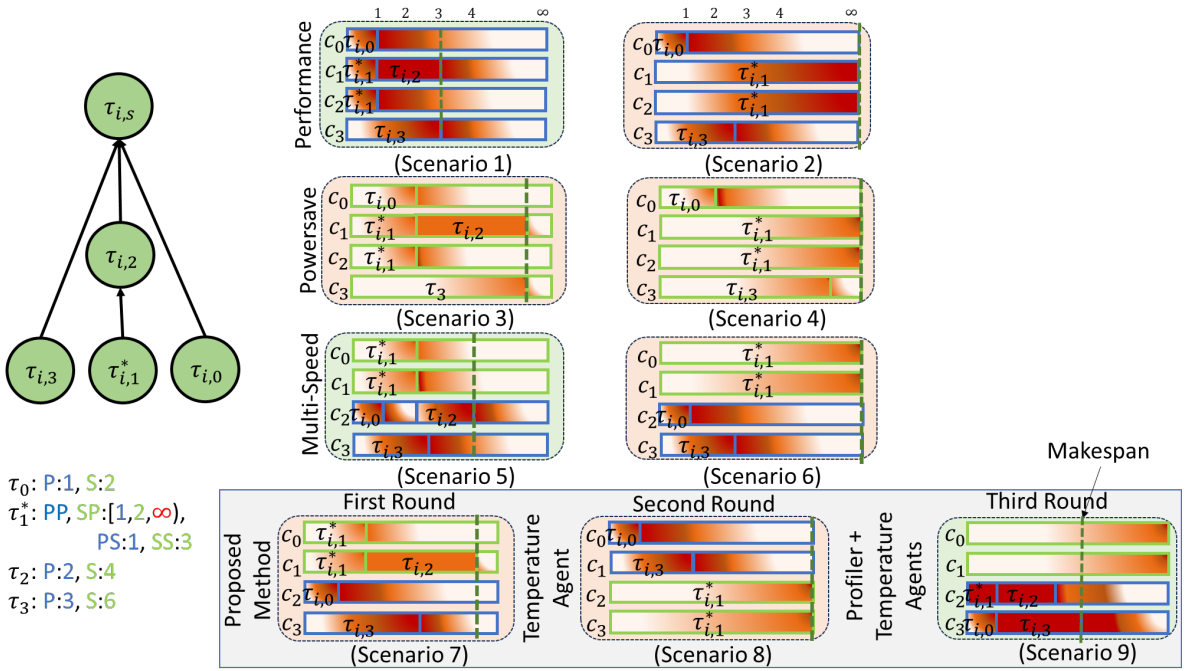


Figure: Scenarios in a Four-Core Architecture: Blue (`performance`), Green (`powersave`). Numbers show execution time. HMARL optimizes allocation.